Deploying RTK CRM

English version of ../DEPLOYMENT.md.

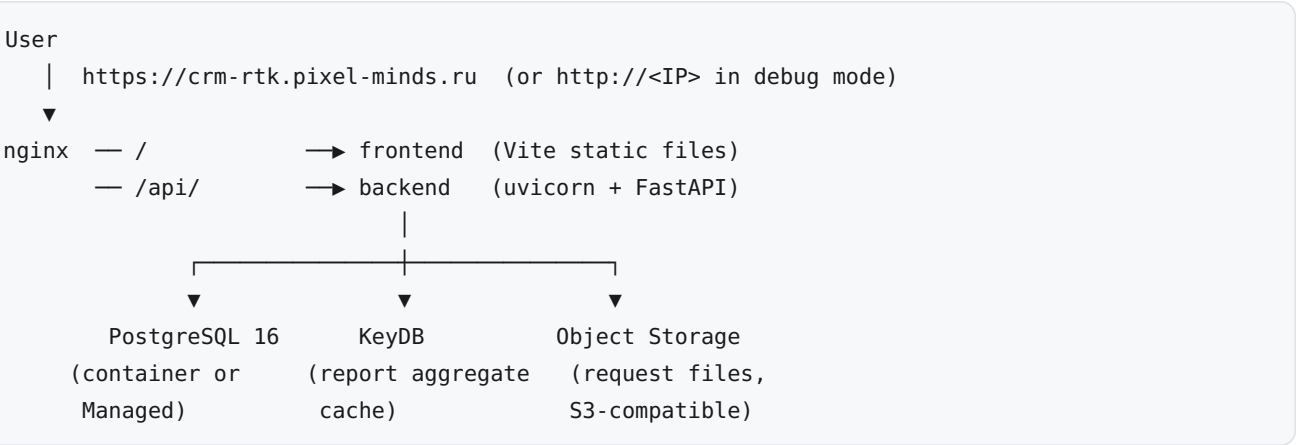
The application is not tied to a specific cloud: the database, the object storage, the cache and the list of allowed origins are set by environment variables. The same build is deployed in both supported contours.

Contour	Composition	Purpose
1. Yandex Cloud (main)	Compute VM + nginx + PostgreSQL + Object Storage + KeyDB	The target contour: data in Russia, 152-FZ, FSTEC order No. 117
2. Render + Vercel + Supabase (alternative)	Render (backend), Vercel (frontend), Supabase (PostgreSQL)	A quick public stand without infra-structure of your own
3. Locally	Docker Compose or SQLite + Vite	Development and tests

- 1. Yandex Cloud
- 2. Render + Vercel + Supabase
- 3. Local run
- 4. Common to all contours

1. Yandex Cloud (the main contour)

Everything is deployed on a single VM: PostgreSQL, KeyDB, backend (FastAPI), frontend (React/Vite) and nginx. The step-by-step VM instructions are in `infra/yandex-cloud/README.md`.



All containers share one docker network; only nginx is exposed (80/443). The backend, the database and the cache are unreachable from outside.

1.1. Recommended VM resources

Parameter	Value
vCPU	2 vCPU, **100%** of the performance level
RAM	4 GB
Disk	30 GB
IP	static (needed for the A record and the certificate)
Image	Ubuntu 22.04 LTS

Measurements during image builds: the memory peak of `npm ci && tsc && vite build` is 1.69 GB, `pip install` of the backend 0.23 GB, and the runtime of the whole stack 143 MiB at rest and about 210 MiB under a load of 50 users. 4 GB is enough with a margin.

The vCPU share matters separately: the load profile from `tests/load/` saturates roughly 1.6 cores (backend ~105%, PostgreSQL ~57%). At a 20% share this profile will not show the design figures — for a demonstration switch to 100% (it changes without recreating the VM).

1.2. Installing on the VM

```
# on the VM, after cloning the repository
cd infra/yandex-cloud
sudo ./setup-vm.sh
```

The script installs Docker and certbot, configures UFW and swap, and copies `.env.prod.example` to `.env`. For a private repository a `GITHUB_TOKEN` or a deploy key is needed.

Then fill in `.env` (the template is `infra/yandex-cloud/.env.prod.example`):

```
DOMAIN=rtk-crm.ru
TLS_ENABLED=true

# Option A: Managed PostgreSQL
# DATABASE_URL=postgresql+asyncpg://rtk_user:<password>@c-<cluster-id>.rw.mdb.yandex-
cloud.net:6432/rtk_crm?ssl=require
# Option B: PostgreSQL in a container on the VM
DATABASE_URL=postgresql+asyncpg://rtk_user:<password>@postgres:5432/rtk_crm

# Report aggregate cache
REDIS_URL=redis://keydb:6379/0

# Request files: Object Storage
S3_ENDPOINT=storage.yandexcloud.net
S3_SECURE=true
S3_REGION=ru-central1
S3_ACCESS_KEY=<yandex-access-key>
S3_SECRET_KEY=<yandex-secret-key>
S3_BUCKET=rtk-crm-files

SECRET_KEY=<a unique string of at least 32 characters>
CORS_ORIGINS=https://rtk-crm.ru
SEED_DEMO_DATA=true

# Card summary (FT-6). Without a key the feature is off: 503 in the API and
# an explanation in the "Summary" tab. The Ministry of Digital Development CA
# certificate is baked into the image, so GIGACHAT_CA_BUNDLE need not be set.
GIGACHAT_CREDENTIALS=<authorization-key-base64>
GIGACHAT_SCOPE=GIGACHAT_API_B2B
GIGACHAT_MODEL=GigaChat
GIGACHAT_FALLBACK_ENABLED=true
```

The variable names must match those read by `backend/core/config.py` (`SECRET_KEY`, `SEED_DEMO_DATA`, `CORS_ORIGINS`, `S3_*`). `ENVIRONMENT=production` enables the `SECRET_KEY` length check at start-up — with a weak key the application does not come up.

Starting:

```
docker compose -f infra/yandex-cloud/docker-compose.prod.yml --env-file .env up -d
```

1.3. Two nginx modes

The mode is selected by the `TLS_ENABLED` variable:

TLS_ENABLED	What comes up	When it is needed
`false`	port 80 only, answering by IP	debugging, no domain or certificate yet
`true`	80 (redirect) + 443 with a certificate	the demonstration to the jury

In mode `true` the certificate must be issued **before** nginx starts: `certbot --standalone` takes port 80, so it will not work while nginx is running. The certificate can also be obtained through Yandex Certificate Manager if the domain is delegated to Yandex Cloud DNS.

1.4. Managed services (optional)

Instead of containers on the VM, managed services can be connected — no code changes, only variables:

Service	Variable	Value
Yandex Managed PostgreSQL	`DATABASE_URL`	a host like `c-<id>.rw.mdb.yandex-cloud.net:6432`, `?ssl=require`
Yandex Object Storage	`S3_ENDPOINT`, `S3_SECURE`, `S3_REGION`	`storage.yandexcloud.net`, `true`, `ru-central1`
Yandex Managed Redis	`REDIS_URL`	the cluster address with TLS

Managed PostgreSQL requires `NullPool` (already configured in `backend/db/session.py`): an external pooler in transaction pooling mode must not hold application connections.

1.5. Backups

`infra/yandex-cloud/backup.sh` is installed into `/etc/cron.daily/rtk-backup` by `setup-vm.sh` and performs a `pg_dump` once a day.

---

2. Render + Vercel + Supabase (the alternative contour)

A contour without infrastructure of your own: the frontend is static files on Vercel, the backend is a container on Render, the database is Supabase PostgreSQL.

```
Vercel (frontend, Vite static files)
   │  https://<project>.vercel.app
   ▼
Render (backend, Docker: uvicorn + FastAPI)
   │  https://<service>.onrender.com
   ▼
Supabase PostgreSQL (Session Pooler, port 5432)
```

The difference from the VM contour: the frontend and the backend live on different domains, so requests are cross-origin and `CORS_ORIGINS` is mandatory, while `VITE_API_URL` points at the backend's absolute address.

2.1. Supabase (database)

The connection string (Session Pooler, port 5432):

```
DATABASE_URL=postgresql+asyncpg://<user>:<password>@<host>.pooler.supabase.com:5432/postgres
```

Requirements:

- the `postgresql+asyncpg://` prefix (not `postgresql://`);
- port `5432` (Session Pooler — it keeps a persistent connection, which asyncpg needs);
- SSL is enabled by default, no extra parameters are required.

> Passwords are never committed to the repository. Values live only in Render → Environment and in the local `backend/.env`.

2.2. Render (backend)

Type: **Web Service** (Docker).

The repository contains a ready Blueprint `render.yaml`: it defines the service, the region, the Dockerfile path and the environment variables, so in the Render panel it is enough to connect the repository and fill in the secrets (`sync: false`). To create the service manually, the parameters are the same:

Parameter	Value
Repository	`ForgeLabHub/ForgeRTKCRM`
Branch	`main`
Environment	**Docker**
Dockerfile Path	`backend/Dockerfile`
Docker Context Directory	`backend`
Region	**Frankfurt (EU Central)** — closer to Supabase eu-west-1
Instance Type	Free

Environment variables (Render → Environment):

Key	Value
`DATABASE_URL`	the value from Supabase
`SECRET_KEY`	a unique string of at least 32 characters
`ENVIRONMENT`	`production`; enables HSTS and the strict `SECRET_KEY` check
`CORS_ORIGINS`	`https://<project>.vercel.app,http://localhost:5173`
`CORS_ORIGIN_REGEX`	a regex for preview hosts (optional)
`SEED_DEMO_DATA`	`true`
`MOCK_MODE`	`true`
`GIGACHAT_CREDENTIALS`	the authorization key from the GigaChat Developers console
`GIGACHAT_SCOPE`	`GIGACHAT_API_B2B`
`GIGACHAT_MODEL`	`GigaChat`
`GIGACHAT_FALLBACK_ENABLED`	`true` — a demo summary instead of 502 when the service fails

> The Ministry of Digital Development CA certificate is baked into the image (`backend/Dockerfile`) and lands in `/certs`, so no additional TLS configuration is needed on Render: `GIGACHAT_CA_BUNDLE` already points at that path by default. If `GIGACHAT_CREDENTIALS` is not set, the "Summary" tab in the card reports that the service is not configured (response 503).
>
> `ENVIRONMENT` must be exactly `production` (or `prod`). With any other value HSTS is not sent and the `SECRET_KEY` length check is not applied.

`backend/start.sh` is a helper script for deploying without Docker (set as the Start Command); it is not built into the image. It performs the same steps: `create_tables()` → `ensure_schema()` → `seed_reference()`.

2.3. Vercel (frontend)

Parameter	Value
Framework Preset	Vite
Root Directory	`frontend`
Build Command	`npm run build`
Output Directory	`dist`

Key	Value
`VITE_API_URL`	`https://<service>.onrender.com`

The frontend is an SPA on React Router, so a rewrite of all paths to `index.html` is needed (otherwise `/reports`, `/directories` and `/settings` return 404). The file `frontend/vercel.json` is already in the repository:

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

2.4. Checking the contour

```
GET https://<service>.onrender.com/           -> {"service":"RTK CRM",...}
GET https://<service>.onrender.com/health     -> {"status":"ok"}
GET https://<service>.onrender.com/api/health -> {"status":"ok","app":"RTK CRM","database":"ok"}
GET https://<service>.onrender.com/healthz    -> {"status":"ok"}
GET https://<service>.onrender.com/readyz     -> {"status":"ok","checks":{...}}
GET https://<service>.onrender.com/metrics    -> Prometheus metrics
GET https://<service>.onrender.com/docs       -> Swagger UI
```

2.5. Typical errors

Symptom	Cause	Solution
Build: `backend/ $` / a pre-deploy error	`Pre-Deploy Command` is filled in on Render	Clear the field
`relation "users" does not exist`	The schema was not created	Check that the start went through `start.sh`/CMD
`connection timeout` to Supabase	Port `6543` (transaction pooler) with asyncpg	Use `5432` (session pooler)
A CORS error in the browser	`CORS_ORIGINS` without the Vercel domain	A comma-separated string without spaces
`Invalid (interleaved) UTF-8` in the logs	`PYTHONIOENCODING` is missing	Already set in the Dockerfile
Slow responses	Render in the US, Supabase in the EU	Set the Render region to Frankfurt

3. Local run

3.1. Through Docker Compose

```
docker compose up -d --build
# frontend: http://localhost:5173
# backend:  http://localhost:8000/docs
```

3.2. Without Docker

```
# Backend (SQLite, no external database needed)
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r backend/requirements.txt
# .env in the repository root: DATABASE_URL=sqlite+aiosqlite:///./dev.db
python scripts/run_dev.py

# Frontend
cd frontend
npm install
npm run dev
```

---

4. Common to all contours

4.1. What happens at start-up

1. The `Dockerfile` installs the dependencies from `backend/requirements.txt` and runs `uvicorn app.main:app --host 0.0.0.0 --port 8000` (the code is in `/app`, `PYTHONPATH=/`).
2. At start-up (`app.main:lifespan`) the database schema is created (`create_tables()`) and **reconciled with the models** (`ensure_schema()`: adds missing columns and indexes, replaces outdated global UNIQUE constraints with composite ones by `scope`), then the reference data is loaded idempotently (`seed_reference()`: 14 B2B stages + 4 B2C stages, users, directions, products).
3. If `SEED_DEMO_DATA=true`, demo universities and interactions are additionally filled in at start-up (only for an empty database).

> The schema reconciliation is needed because `create_all(checkfirst=True)` does not alter existing tables: a database created by an earlier version of the application silently stayed without `scope`, and loading the demo data failed with `column workflow_stages.scope does not exist`. `ensure_schema()` brings such a schema up to the models without deleting data and marks the Alembic head revision so that a subsequent `alembic upgrade head` does not try to create tables that already exist.

4.2. Authentication modes

Mode	`MOCK_MODE`	What is needed	Usage
JWT + database	`true`	Only the database and `SECRET_KEY`	The hackathon contour, both clouds
Keycloak	`false`	Keycloak, a realm, a client and a token adapter	The customer's closed contour

The API currently implements JWT authentication through `POST /api/auth/login`. The value `false` is reserved for Keycloak and does not switch the backend automatically.

4.3. Demo access

Role	E-mail	Password
Administrator	`admin@rtk.ru`	`admin123`
Executive	`manager@rtk.ru`	`manager123`
KAM	`kam@rtk.ru`	`kam123`

4.4. Security

- Database passwords and `SECRET_KEY` live only in environment variables.
- `backend/.env` is in `.gitignore`; the repository holds only `.env.example`.
- JWT access to all endpoints except `/health` and the login.
- After a public demonstration change the database password and revoke the access tokens that were used.